

Explainable Anomaly Detection Method Using LLM Embeddings Model with SHAP

Taisei Matsumura

INIAD, Toyo University

Tokyo, Japan

s1f102101974@iniad.org

Satoshi Okada

INIAD, Toyo University

Tokyo, Japan

satoshi.okada@iniad.org

Koki Watarai

INIAD, Toyo University

Tokyo, Japan

koki.watarai@iniad.org

Takuho Mitsunaga

INIAD, Toyo University

The Tokyo Foundation for Policy Research

Tokyo, Japan

takuho.mitsunaga@iniad.org

Abstract—In recent years, the black-box nature of AI models in anomaly detection using log data has raised concerns about reliability and transparency. Conventional text-based anomaly detection methods have relied on structuring logs using log parsers and extracting features with techniques such as Term Frequency-Inverse Document Frequency (TF-IDF) and word2vec. How the AI model determines whether abnormal logs or not provides valuable clues for responding after detection. However, these methods depend on word frequency, making it difficult to explain the basis for anomaly detection decisions clearly. In this study, we propose a method to increase the transparency and interpretability of the model and clarify the criteria for anomaly detection by identifying features to contribute using Shapley Additive Explanations (SHAP). Our proposed method uses vectorized log data event templates using OpenAI Embeddings, which can be visually represented while achieving a high detection rate. To our knowledge, this study contributes to improving the interpretability and transparency of models by utilizing the Large Language Models (LLM) Embeddings Model for feature extraction, plotting them in a reduced-dimensional coordinate space using principal component analysis (PCA), and enhancing explainability using SHAP.

Index Terms—Anomaly Detection, LLM, Explainable AI(XAI), SHAP

I. INTRODUCTION

System logs are detailed records of system operations and serve as an essential data source for anomaly detection. By analyzing these logs, cyberattacks and system failures can be detected early, making them crucial data for maintaining system stability and security. As cybersecurity threats become more sophisticated, the importance of anomaly detection techniques based on log data is increasing.

Anomaly detection, particularly in text-based approaches, often involves structuring unstructured logs using log parsers such as Drain [1] and extracting features through techniques such as TF-IDF [2] and word2vec [3]. These approaches achieve a certain level of anomaly detection accuracy. However, since they rely on word frequency, it is difficult to clearly explain the basis for the decisions in the anomaly logs they detect.

The black-box nature of AI models, especially in anomaly detection, makes it crucial to understand why a model classified a log as anomalous. This understanding is essential for improving model performance and ensuring the reliability and accountability of its predictions. To address these concerns,

Explainable Artificial Intelligence (XAI) has emerged as a critical technology. XAI techniques, such as Shapley Additive Explanations (SHAP), provide insights into AI models' decision-making processes, allowing for identifying features that contribute to specific outcomes.

Moreover, the rapid evolution of Large Language Models (LLMs) in recent years has led to the emergence of advanced embedding models, which have brought significant advancements in the fields of Natural Language Processing (NLP) and machine learning. These embedding models, such as those provided by OpenAI, enable the transformation of textual data into high-dimensional vector representations, facilitating more effective feature extraction and analysis. As these technologies evolve, their applications expand to various domains, including anomaly detection, garnering significant attention.

In this study, we propose a novel approach to enhance the transparency and interpretability of anomaly detection models. We leverage LLM-based embedding models for feature extraction and vectorized log data event templates and then apply Principal Component Analysis (PCA) to reduce the dimensionality of these vectors. This dimensionality reduction allows for visualizing log data in a lower-dimensional coordinate space. Furthermore, we employ SHAP to analyze and clarify the contributions of individual features to the model's predictions, thereby elucidating the criteria for anomaly detection. We used the BlueGene/L dataset from Log-hub to validate this approach and applied three different machine learning models—Decision Tree, Random Forest, and Gradient Boosting—to evaluate its performance.

Our proposed method contributes to increasing the interpretability and transparency of the model, clarifying the basis for decisions, and providing valuable information for responding after anomaly detection, in addition to improving the accuracy of anomaly detection. By integrating state-of-the-art embedding models with XAI techniques, this research aims to make AI models more transparent and explainable, particularly in critical cybersecurity and system monitoring applications.

II. RELATED WORKS

Log anomaly detection has been extensively studied by numerous researchers.

Wang et al. [4] proposed a method called LogEvent2vec for anomaly detection based on log data. This method constructs log sequence vectors from event IDs and uses these vectors as features to build an anomaly detection model. In their research, similar to the approach proposed in this study, the BlueGene/L (BGL) dataset was utilized, and a combination of the bary vectorization technique and Random Forest was employed. Their experimental results demonstrated that their model achieved high accuracy, with an F1 score of 0.88 and an AUC of 0.97. However, Wang et al.'s method has limitations in terms of model explainability and transparency. Specifically, it is challenging for users to understand the specific reasoning behind how the vectors generated by LogEvent2vec on the interpretability of their approach.

Alam et al. [5] proposed a framework called SXAD (Shapely eXplainable AI-Based Anomaly Detection) to enhance the explainability by utilizing the Hadoop Distributed File System (HDFS) dataset and TF-IDF. They built an anomaly detection model by extracting features from log data event templates using TF-IDF and mapping these features to event IDs. They achieved an exceptionally high accuracy of 0.999 by employing SHAP to identify the event IDs contributing to the anomalies. Alam et al.'s method shares some similarities with our proposed method in that it addresses the black-box nature of models by using SHAP on text-based anomaly detection. However, we utilize OpenAI Embeddings for feature extraction to enhance the explainability.

Xiao et al. [6] proposed a method called Loader for anomaly detection in log data. Loader is a semi-supervised learning approach that utilizes the Transformer architecture to learn normal log patterns and detect anomalies. In experiments using the HDFS dataset, it achieved an F-measure of 0.978, and in the BGL dataset, it achieved an F-measure of 0.933. Xiao et al.'s method has limitations in terms of model explainability and transparency. Specifically, when Loader uses the Transformer architecture to detect anomalies in log data, it is challenging for users to understand the specific reasoning behind how the vectors are interpreted.

Explainable Artificial Intelligence (XAI) is a crucial technology that helps understand the predictions made by DL models and ensures accountability. In a study by Okada et al. [7], a new method is proposed that utilizes XAI to generate adversarial examples (AEs) against DL-based Network Intrusion Detection Systems (NIDS). This method identifies important features and visualizes them by plotting them in a 3D space. The experimental results demonstrate that the generated AEs successfully evade NIDS detection with a high probability, indicating that the perturbations in the problem space were effectively implemented.

Briola et al. [8] conducted a study on the integration of Explainable Artificial Intelligence (XAI) in breast cancer diagnosis, focusing on the use of Federated Learning (FL) to balance privacy protection and model performance. They used the Wisconsin Breast Cancer Dataset and the Wisconsin Diagnostic Breast Cancer Dataset to demonstrate that FL can maintain high accuracy and explainability while safeguarding

privacy. Additionally, they proposed a method to enhance the reliability of breast cancer diagnosis and treatment by utilizing Shapely values for explainability.

Bansal et al. [9] conducted a study on the ethical implementation of artificial intelligence (AI) in business planning, with a particular focus on ensuring fairness and transparency. The study highlights that while AI adoption enhances corporate efficiency, concerns about fairness and transparency as ethical issues are increasingly prominent. They explored approaches to maintaining fairness in AI systems, reducing bias, and improving transparency, and analyzed how these factors influence business decision-making. Additionally, they examined methods to enhance AI transparency by identifying the factors influencing AI decisions using Shapely values.

III. PROPOSED METHOD

The overview of our method is shown in Figure 1. The detailed process of each step is described below.

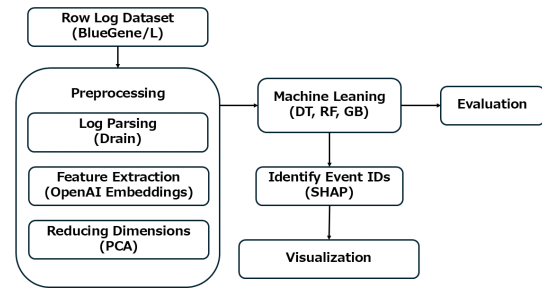


Fig. 1: Flowchart of Proposed Method

A. Log Parsing

Since unstructured log data is difficult to analyze and classify, it is important to extract log events and event templates as an initial step in the analysis process. In this study, we employed Drain [1] for log parsing to extract event templates from unstructured logs. Drain is a log parsing tool that uses a fixed-depth parse tree to analyze log entries based on user-defined parameters, transforming each entry into a specific event. It analyzes log entries by transforming them into templates.

B. Feature Extraction Using Embeddings Model

In this study, we used OpenAI's embedding model to vectorize and extract features from event templates in system logs. Specifically, we used the text-embedding-3-small model to convert event templates into 1536-dimensional vectors. This model balances computational efficiency and accuracy, making it suitable as the basis for an anomaly detection model.

OpenAI's latest model, text-embedding-3-large [10], generates 3072-dimensional embeddings and achieves a score of 64.6% on the Massive Text Embedding Benchmark (MTEB) [11]. However, it requires significant memory and processing resources, which can make it difficult to run in

resource-limited environments, especially on cloud or on-premise servers.

On the other hand, the text-embedding-3-small model is relatively lightweight, achieves high computational efficiency while maintaining a score of 62.3%, and operates stably even in resource-constrained environments. This makes it possible to build an anomaly detection model that maintains performance while reducing computational load.

C. Reducing Embedding Dimensions

The high-dimensional vectors (1536 dimensions) are reduced to two dimensions using Principal Component Analysis (PCA) and used as inputs for the anomaly detection model and for visualization to capture the characteristics of each event template in a coordinate space. Figure 2 shows similar event templates visualized in the same space, enabling both model learning and visual interpretation of event clustering and anomaly patterns. The results of the dimensionality reduction reveal a tendency for similar event templates to be plotted close within the coordinate space. This suggests that the OpenAI Embeddings model appropriately captures the relative similarity between event templates and maintains this relationship even after dimensionality reduction using PCA. These results confirm that the proposed method effectively represents the characteristics of event templates, allowing for analysis that considers the semantic similarity of templates during anomaly detection.

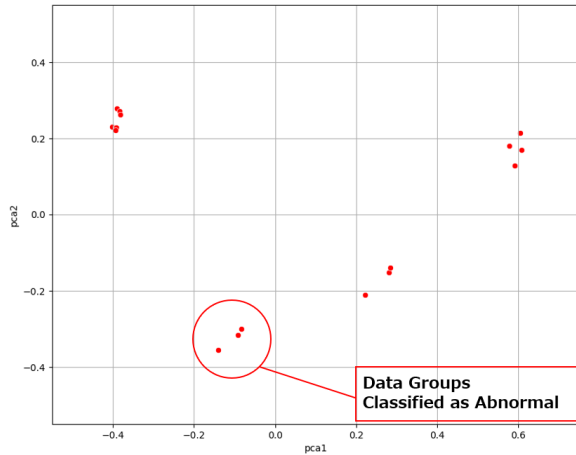


Fig. 2: Results of PCA analysis on event templates

D. Anomaly Detection

We used Decision Tree (DT), Random Forest (RF), and Gradient Boosting (GB) as machine learning models for anomaly detection in system logs. By employing these methods, we aimed to analyze the features of system logs from different approaches and improve the accuracy of anomaly detection.

E. Identification of Event IDs Using SHAP

We aim to improve the explainability and transparency of the model by utilizing an XAI method, SHapley Additive ex-Planations (SHAP) [12] to identify the event IDs contributing

to the model's output. SHAP is a widely used interpretability method in machine learning that efficiently computes Shapley values [5] [7] [8] [9]. Given the vast amount of data in system logs, calculating Shapley values across the entire dataset was not feasible due to computational constraints. The system resources available for this study, including OS, memory, and CPU specifications, were insufficient to efficiently process the full dataset required for SHAP analysis. To address this limitation, we balanced computation time and interpretability by calculating Shapley values on a representative 10% sample of the test data. Using a 10% sample allows effective analysis within our resource constraints. While analyzing the full dataset might yield more insights, this approach offers a practical balance within the available computational resources.

F. Visualization

Throughout the process, we identified the event IDs that contributed to the model's output using SHAP to enhance the model's explainability and transparency. Finally, we visualized the results using SHAP's bar plot. This visualization allows us to understand how the model detects anomalies.

IV. EVALUATION

A. Data Set

We use the BlueGene/L (BGL) dataset [13] to evaluate the proposed method, an open logs dataset collected from the BlueGene/L supercomputer system at Lawrence Livermore National Laboratory (LLNL) [14]. Because this dataset contains alert and non-alert messages identified by tags and is suitable for studies on alert detection and prediction, it has been used in many studies on log analysis, anomaly detection, and fault prediction. The BGL dataset contains 4,747,963 log messages and 348,460 anomaly log messages.

B. Performance Metrics

The evaluation results are categorized as follows.

- **True Positive (TP)**: Number of instances correctly identified as positive.
- **False Positive (FP)**: Number of instances incorrectly identified as positive.
- **True Negative (TN)**: Number of instances correctly identified as negative.
- **False Negative (FN)**: Number of instances incorrectly identified as negative.

This paper evaluated the prediction using accuracy, precision, recall, and F1 score, which are defined by the following equations.

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + TN + FN} \quad (1)$$

$$\text{Precision} = \frac{TP}{TP + FP} \quad (2)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (3)$$

$$F1 \text{ Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (4)$$

- **Accuracy:** The percentage of the total number of instances that are correctly classified, including both positive and negative instances.
- **Precision:** The percentage of instances that are correctly identified as positive out of the total instances identified as positive.
- **Recall:** The percentage of actual positive instances that are correctly identified as positive.
- **F1 Score:** The harmonic mean of precision and recall, providing a balance between them.

C. Experimental Settings

The following procedures were used to evaluate the results.

- 1) Set 80% of the BGL dataset as training data and 20% as test data. Perform machine learning using Decision Tree, Random Forest, and Gradient Boosting.
- 2) Calculate TP (True Positive), FP (False Positive), TN (True Negative), FN (False Negative), accuracy, precision, recall, and F1 score from the prediction results.
- 3) For the completed models, calculate the SHAP values for the event IDs that contributed to the output of the anomaly class (class1) using 10% of the test data for each model.
- 4) Based on the identified event IDs, determine the event templates from the PCA-reduced data and plot them as a graph.

Our experiments were conducted under the following hardware specifications. Table I provides an overview of the system components used, offering context for the computational resources available during the analysis.

TABLE I: Hardware Specifications

Component	Specification
Processor	Intel(R) Core(TM) i7-10750H CPU 2.60GHz, 2.59 GHz
Installed RAM	16.0 GB (15.8 GB Usable)
Operating System	64-bit Windows 11

D. Result

Ultimately, the evaluation results of each machine learning model are shown in Table II. The Decision Tree model achieved the highest accuracy, with an Accuracy of 0.9949, Precision of 0.9424, Recall of 0.9908, and an F1 Score of 0.9660. The Random Forest model also demonstrated similar performance, achieving an Accuracy of 0.9949, Precision of 0.9423, Recall of 0.9908, and an F1 Score of 0.9660. On the other hand, the Gradient Boosting model showed slightly lower performance, with an Accuracy of 0.9946, Precision of 0.9412, Recall of 0.9892, and an F1 Score of 0.9646.

These accuracy and precision calculations were derived based on the results shown in Table III.

Using SHAP, we identified the features that contributed to the anomalies in the trained model. Figure 3 illustrates the

TABLE II: Results of Anomaly Detection in Each Model (Accuracy, Precision, Recall, F1 Score)

ML Model	Accuracy	Precision	Recall	F1 Score
Decision Tree	0.9949	0.9424	0.9908	0.9660
Random Forest	0.9949	0.9423	0.9908	0.9660
Gradient Boosting	0.9946	0.9412	0.9892	0.9646

TABLE III: Results of Anomaly Detection in Each Model (TP, FP, TN, FN)

ML Model	TP	FP	TN	FN
Decision Tree	68791	4208	869062	638
Random Forest	68789	4209	869061	640
Gradient Boosting	68676	4291	868979	753

parameters determined to be abnormal using SHAP. In this figure, the vertical axis is event IDs. In the DT model shown on the left, the event IDs are 333, 334, and 39 from the top. This data set stores various events related to the system. 333 is related to the CPU's "Translation Lookaside Buffer (TLB)," 334 is related to the OS's "kernel panic," and 39 is related to a "Lustre file system mount error."

E. Discussion

In Table II, the proposed method achieves a higher F1 Score (0.9660) compared to the study by Wang et al.'s [4] F1 Score (0.88), which used the same dataset. The SHAP analysis further supports this improvement, where the important event IDs—333, 334, and 39—were correctly identified as contributing to anomalies. Table IV illustrates the mapping of these encoded event IDs to their corresponding event templates and labels, all of which indicate anomalies. These results confirm that the proposed method outperforms previous studies and effectively identifies key factors in anomaly detection.

V. CONCLUSION

We proposed an explainable anomaly detection system. It visualizes the features contributing to anomaly detection using the LLM Embeddings model and SHAP in the context of system log anomaly detection. Traditional text-based anomaly detection methods have faced issues with the black-box nature of feature selection and model decision criteria. However, by utilizing SHAP, we were able to clarify the specific features that contribute to the model's decisions and make the results visually comprehensible.

Additionally, we achieved the highest accuracy when using a decision tree model for anomaly detection. The outcomes of this research not only improve the accuracy of anomaly detection in system logs and enhance the transparency of AI model decision processes, thereby increasing reliability in practical applications. Future challenges include applying the proposed method to more diverse log datasets, conducting comparative evaluations with other machine learning models to validate the proposed method's generalization and effectiveness, and exploring feature vectorization using alternative

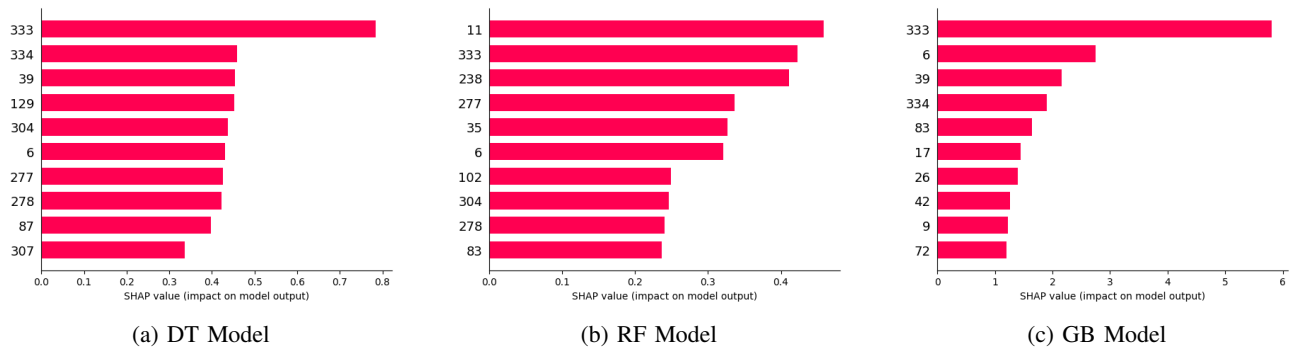


Fig. 3: Encoded Event IDs that Contributed to the Anomaly

TABLE IV: Encoded Event ID, Event Templates, and Labels

Encoded Event ID	Event Templates	Label
333	<*> TLB error interrupt	KERNTLBE
334	kernel panic	KERNPAN
39	Lustre mount FAILED : <*> : block_id : location	KERNMNTF

embedding models to further enhance the robustness of the anomaly detection process. Additionally, we are considering the implementation of an indicator function to track link status throughout the time series across different datasets. This function would enable monitoring of changes and anomalies in link behavior over time, providing more granular insights and improving the accuracy of anomaly detection in heterogeneous data environments.

REFERENCES

- [1] P. He, J. Zhu, Z. Zheng and M. R. Lyu, "Drain: An Online Log Parsing Approach with Fixed Depth Tree," 2017 IEEE International Conference on Web Services (ICWS), Honolulu, HI, USA, 2017, pp. 33-40, doi: 10.1109/ICWS.2017.13.
- [2] C. Sammut and G. I. Webb, "TF-IDF," Encyclopedia of Machine Learning, Boston, MA: Springer US, 2010, pp. 986-987. doi: 10.1007/978-0-387-30164-8_832.
- [3] T. Mikolov, "Efficient estimation of word representations in vector space," arXiv preprint arXiv:1301.3781, 2013.
- [4] Wang, J.; Tang, Y.; He, S.; Zhao, C.; Sharma, P.K.; Alfarraj, O.; Tolba, A. LogEvent2vec: LogEvent-to-Vector Based Anomaly Detection for Large-Scale Logs in Internet of Things. Sensors 2020, 20, 2451. <https://doi.org/10.3390/s20092451>
- [5] K. Alam, K. Kifayat, G. A. Sampedro, V. Karovič and T. Naeem, "SXAD: Shapely eXplainable AI-Based Anomaly Detection Using Log Data," in IEEE Access, vol. 12, pp. 95659-95672, 2024, doi: 10.1109/ACCESS.2024.3425472.
- [6] T. Xiao et al., "Loader: A Log Anomaly Detector Based on Transformer," in IEEE Transactions on Services Computing, vol. 16, no. 5, pp. 3479-3492, Sept.-Oct. 2023, doi: 10.1109/TSC.2023.3280575.
- [7] S. Okada, H. Jmila, K. Akashi, T. Mitsunaga, Y. Sekiya, H. Takase, G. Blanc, and H. Nakamura, "XAI-driven adversarial attacks on network intrusion detectors," in *Proc. 2024 European Interdisciplinary Cybersecurity Conf.*, Xanthi, Greece, 2024, pp. 65-73. doi: 10.1145/3655693.3655714.
- [8] E. Briola, C. C. Nikolaidis, V. Perifanis, N. Pavlidis, and P. Efraimidis, "A Federated Explainable AI Model for Breast Cancer Classification," in *Proc. 2024 European Interdisciplinary Cybersecurity Conf.*, Xanthi, Greece, 2024, pp. 194-201. doi: 10.1145/3655693.3660255.
- [9] A. Bansal, D. A. K. S. Gangadharan, R. R. R. M. M. Ismail and N. K. A. Halaf, "Ethical Considerations of AI Implementation in Business Planning: Ensuring Fairness and Transparency," 2023 International Conference on Innovative Computing, Intelligent Communication and Smart Electrical Systems (ICSES), Chennai, India, 2023, pp. 1-6, doi: 10.1109/ICSES60034.2023.10465461.
- [10] OpenAI. "New Embedding Models and API Updates." OpenAI. <https://openai.com/index/new-embedding-models-and-api-updates>, accessed October 30, 2024.
- [11] N. Muennighoff, N. Tazi, L. Magne, and N. Reimers, "MTEB: Massive Text Embedding Benchmark," in Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics, pages 2014-2037, Dubrovnik, Croatia. Association for Computational Linguistics.
- [12] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in Advances in Neural Information Processing Systems 30, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds. Curran Associates, Inc., 2017, pp. 4765-4774.
- [13] J. Zhu, S. He, P. He, J. Liu and M. R. Lyu, "Loghub: A Large Collection of System Log Datasets for AI-driven Log Analytics," 2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE), Florence, Italy, 2023, pp. 355-366, doi: 10.1109/ISSRE59848.2023.00071.
- [14] A. Oliner and J. Stearley, "What Supercomputers Say: A Study of Five System Logs," 37th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN'07), Edinburgh, UK, 2007, pp. 575-584, doi: 10.1109/DSN.2007.103.